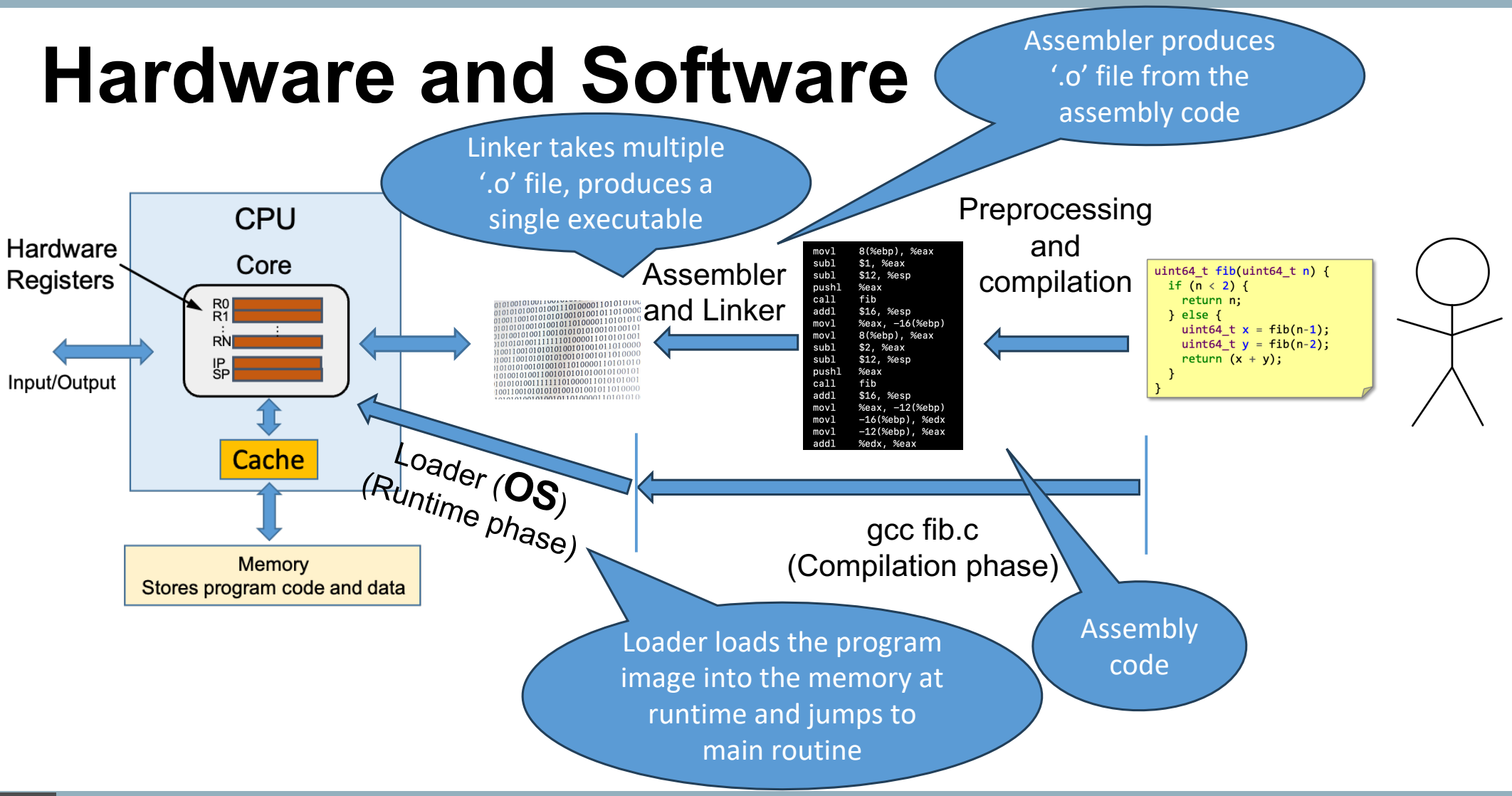
Lecture 11: Mid-Semester Review

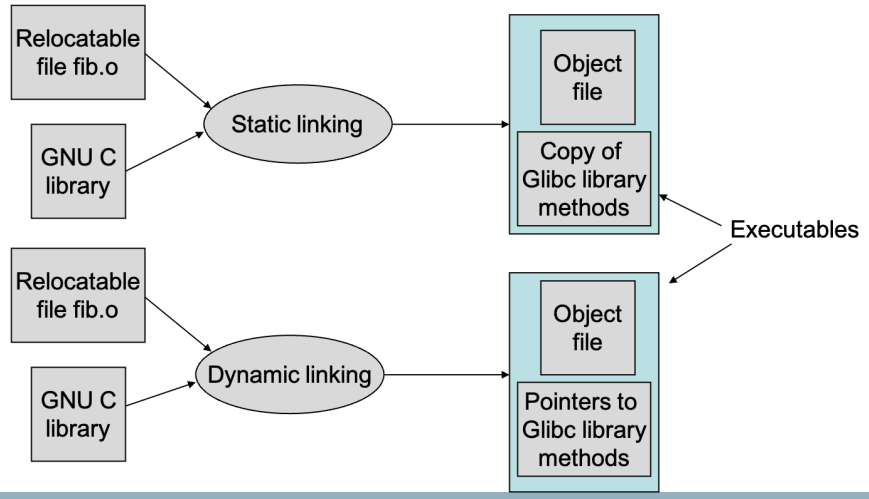
Vivek Kumar
Computer Science and Engineering
IIIT Delhi
vivekk@iiitd.ac.in

Hardware and Software



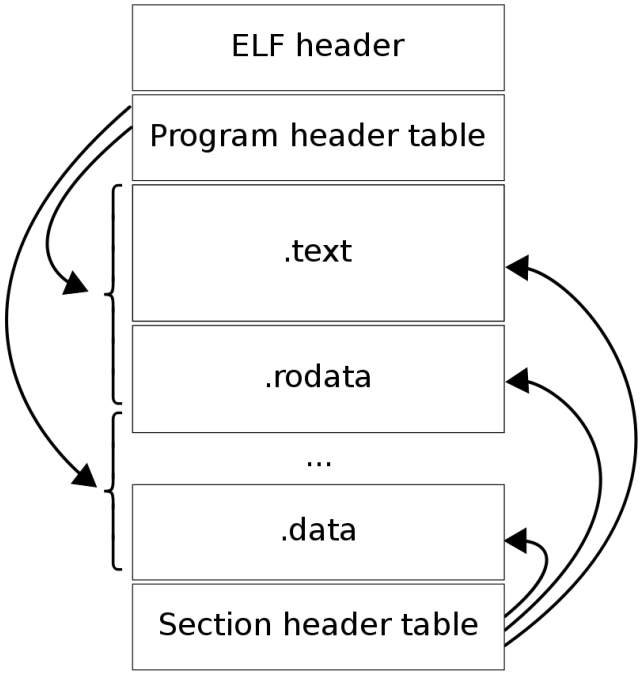
Static and Dynamic Linking

```
$ gcc -static -o fib fib.c
$ file fib
...ELF 64-bit executable...statically linked
$ ls -l fib
-rwxrwxr-x 1 vivek vivek 845304 Aug  6 10:26 fib
$ gcc -o fib fib.c
$ ls -l fib
-rwxrwxr-x 1 vivek vivek 8328 Aug  6 10:27 fib
```



- Static linking
  - Each and every library modules referenced in the relocatable file is copied into the final executable
    - Static binding at compile time
- Dynamic linking
  - Final executable only contains references (pointers) to the library method instead of the copy of a library method
    - Binding with library done at runtime during execution

Executable and Linkable Format (ELF)




```
vivek@possum:~/elf_os-l2$ cat hello.c
#include<stdio.h>
int my_global1 = 1, my_global2[1024*1024];
char* str = "Updated value of my_global1";

int main() {
  my_global1 += 1;
  printf("%s = %d\n", str, my_global1);
  return 0;
}
```

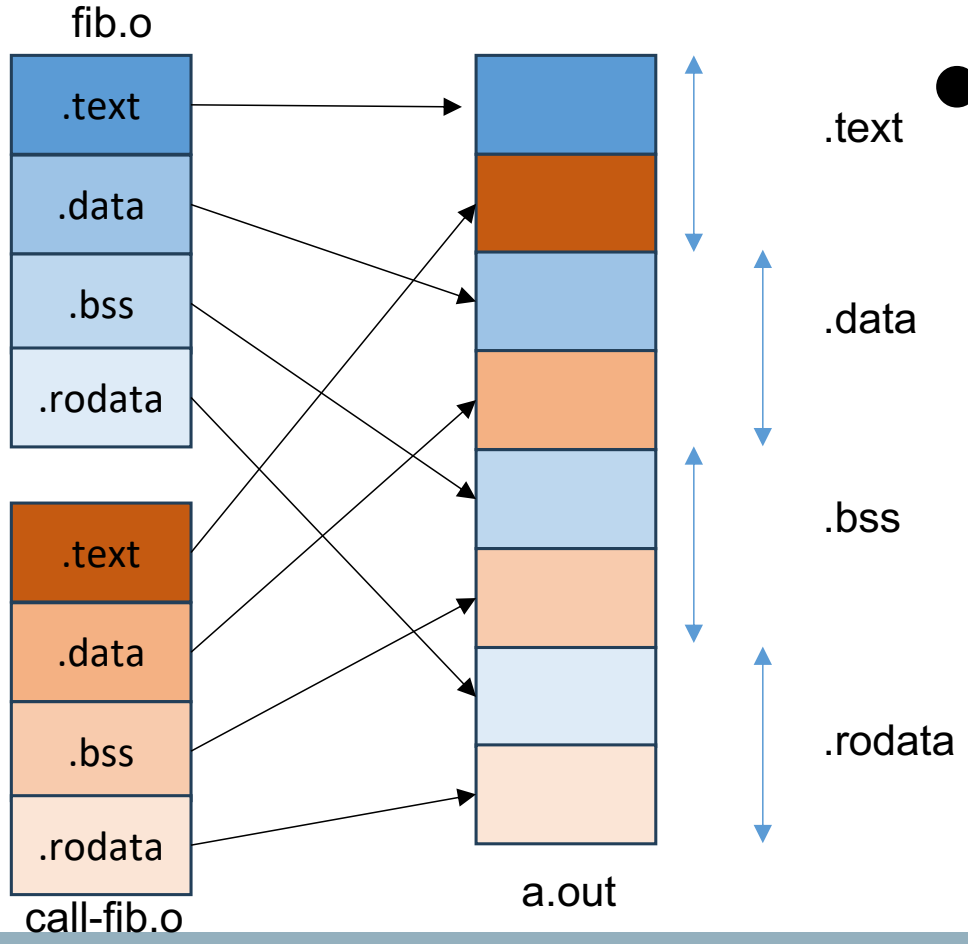
A B C D E F

Image source: https://en.wikipedia.org/wiki/Executable_and_Linkable_Format

Linker: Merges the Sections

```
[vivek@possum]$ cat fib.c
int get_number();

int fib(int n) {
  if(n<2) return n;
  else return fib(n-1)+fib(n-2);
}

int compute() {
  int n = get_number();
  return fib(n);
}
```

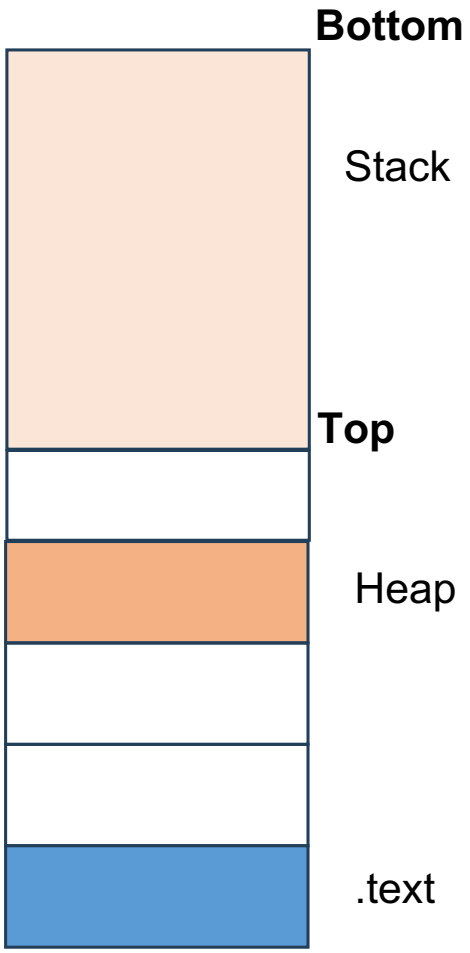
```
[vivek@possum]$ cat call-fib.c
int compute();
int get_number() {
  return 40;
}
int main() {
        int result = compute();
        return 0;
}
```



- Relocation is the process of merging the sections and resolving the symbol addresses

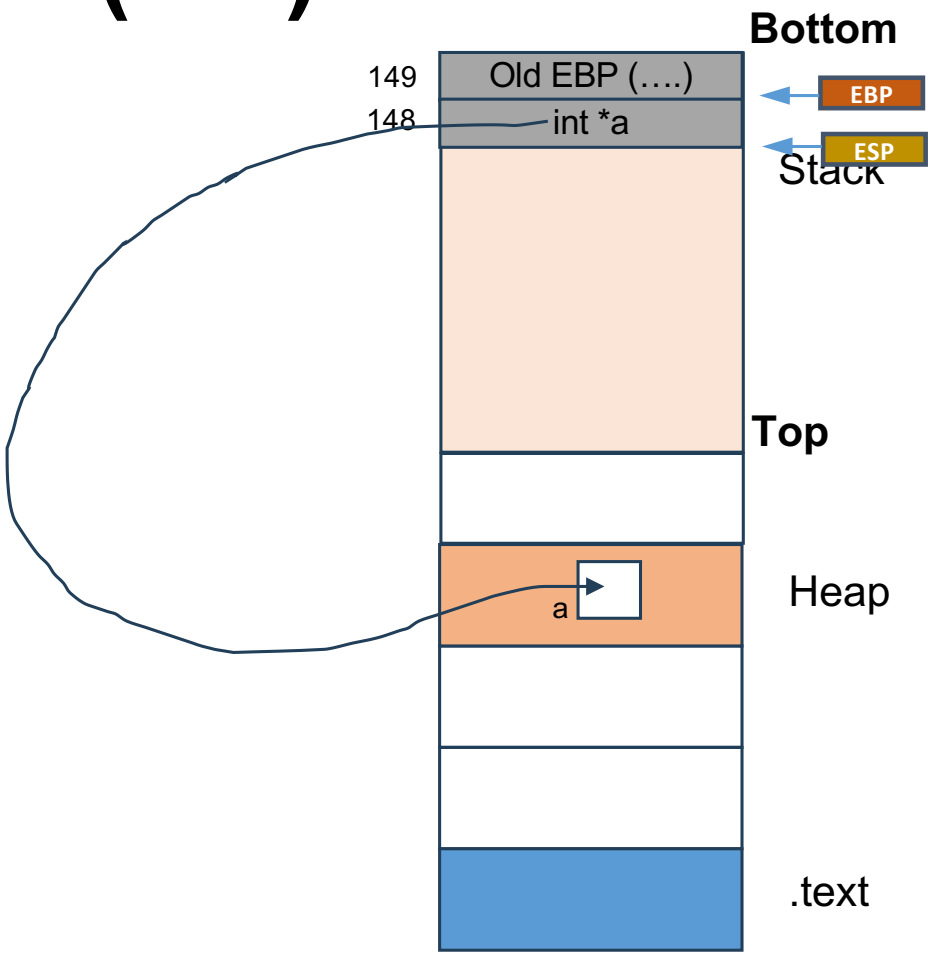
Program Execution (1/6)

```
L1: int g=0;

L2: void main() {
L3:     int *a = (int*) malloc(4);
L4:     char *b = “Hello World”;
L5:     foo(a);
L6:     g=*a;
L7: }

L8: void foo(int* b) {
L9:     *b = 20;
L10:}
```



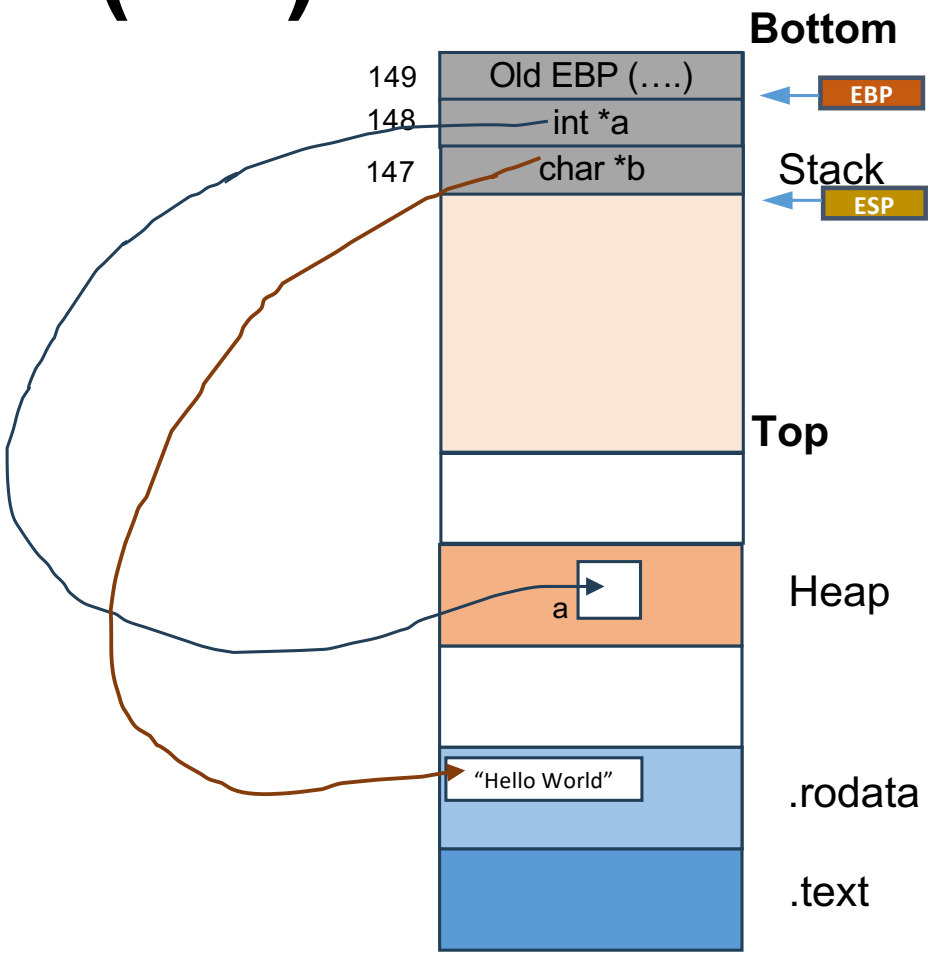
Program Execution (2/6)

```
L1: int g=0;

L2: void main() {
L3:     int *a = (int*) malloc(4);
L4:     char *b = “Hello World”;
L5:     foo(a);
L6:     g=*a;
L7: }

L8: void foo(int* b) {
L9:     *b = 20;
L10:}
```



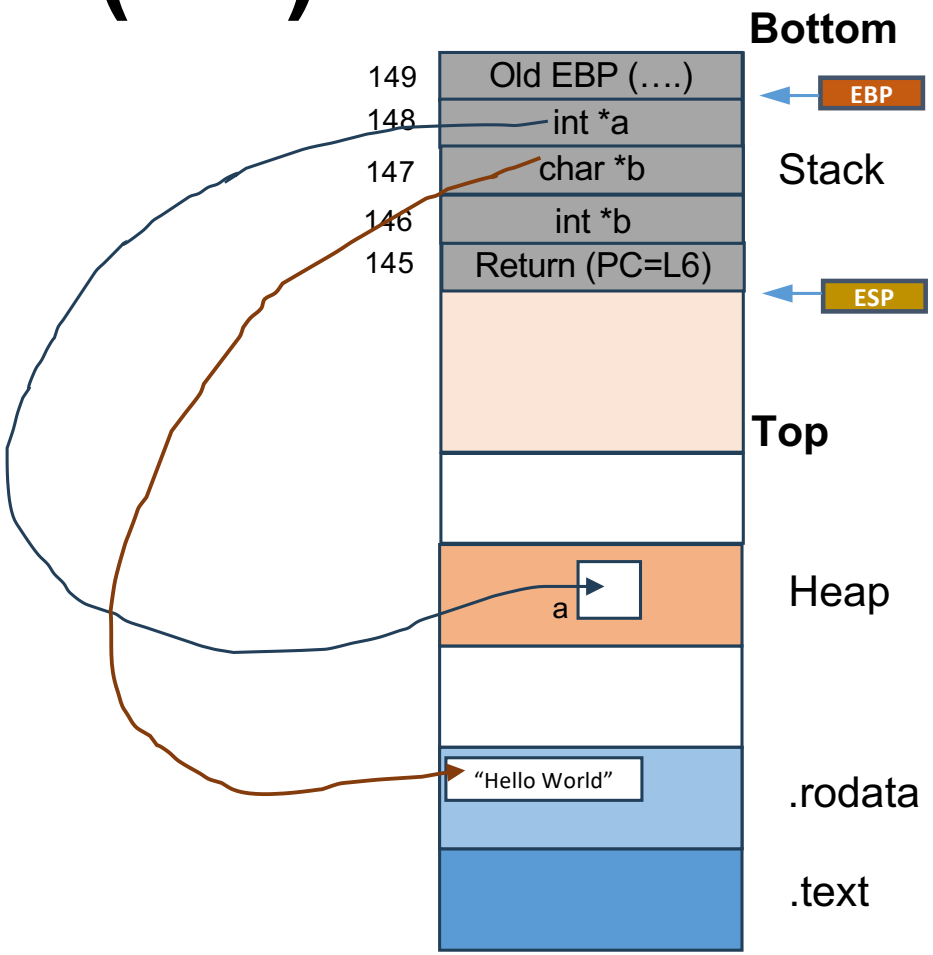
Program Execution (3/6)

```
L1: int g=0;

L2: void main() {
L3:     int *a = (int*) malloc(4);
L4:     char *b = “Hello World”;
L5:     foo(a);
L6:     g=*a;
L7: }

L8: void foo(int* b) {
L9:     *b = 20;
L10:}
```



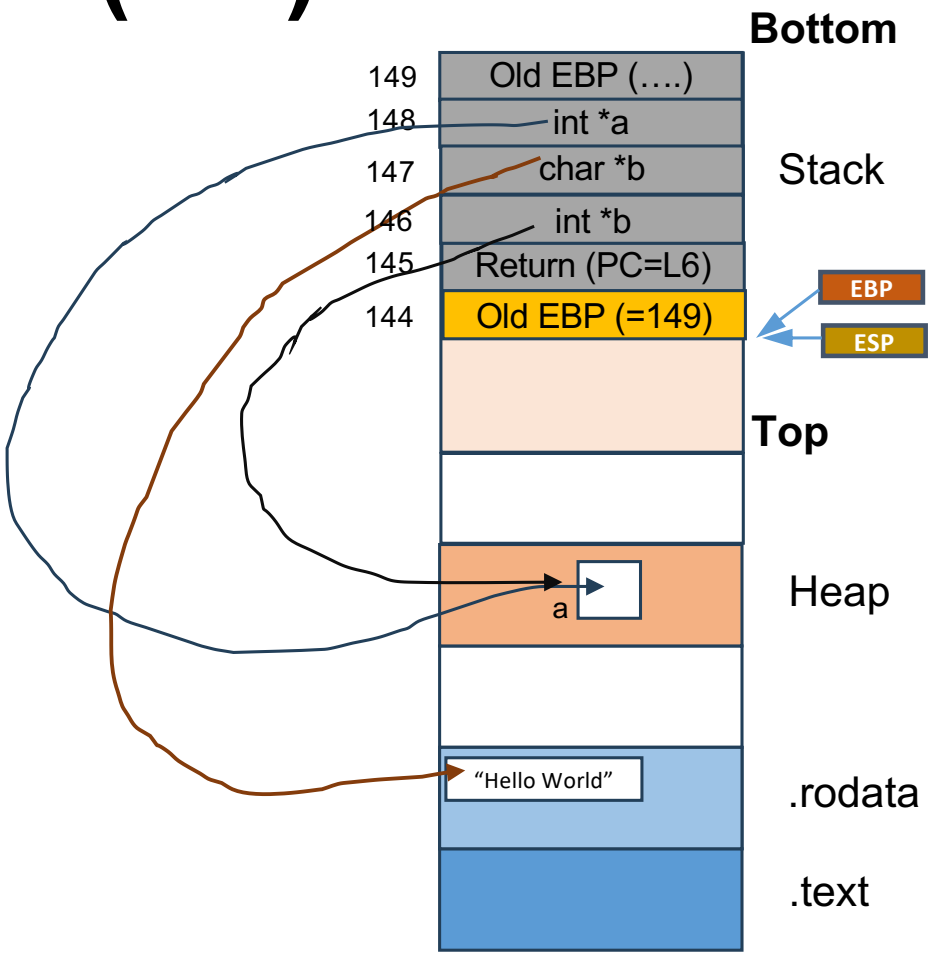
Program Execution (4/6)

```
L1: int g=0;

L2: void main() {
L3:     int *a = (int*) malloc(4);
L4:     char *b = “Hello World”;
L5:     foo(a);
L6:     g=*a;
L7: }

L8: void foo(int* b) {
L9:     *b = 20;
L10:}
```



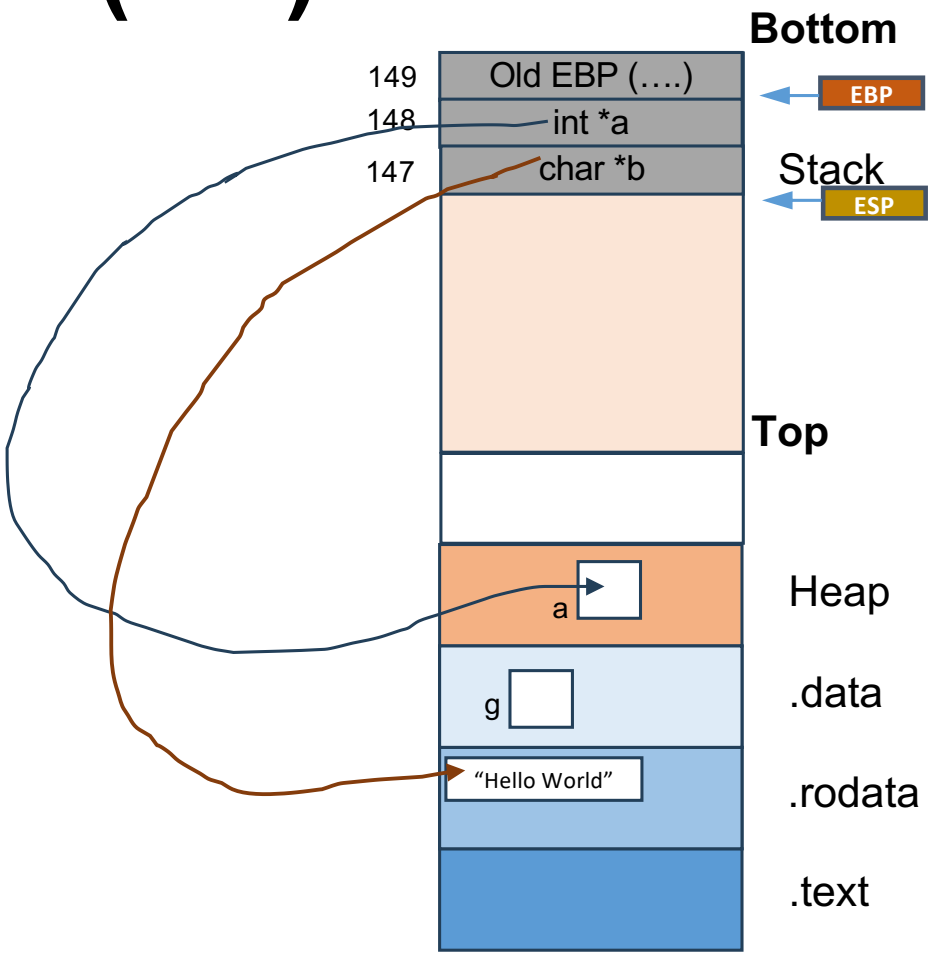
Program Execution (5/6)

```
L1: int g=0;

L2: void main() {
L3:     int *a = (int*) malloc(4);
L4:     char *b = “Hello World”;
L5:     foo(a);
L6:     g=*a;
L7: }

L8: void foo(int* b) {
L9:     *b = 20;
L10:}
```

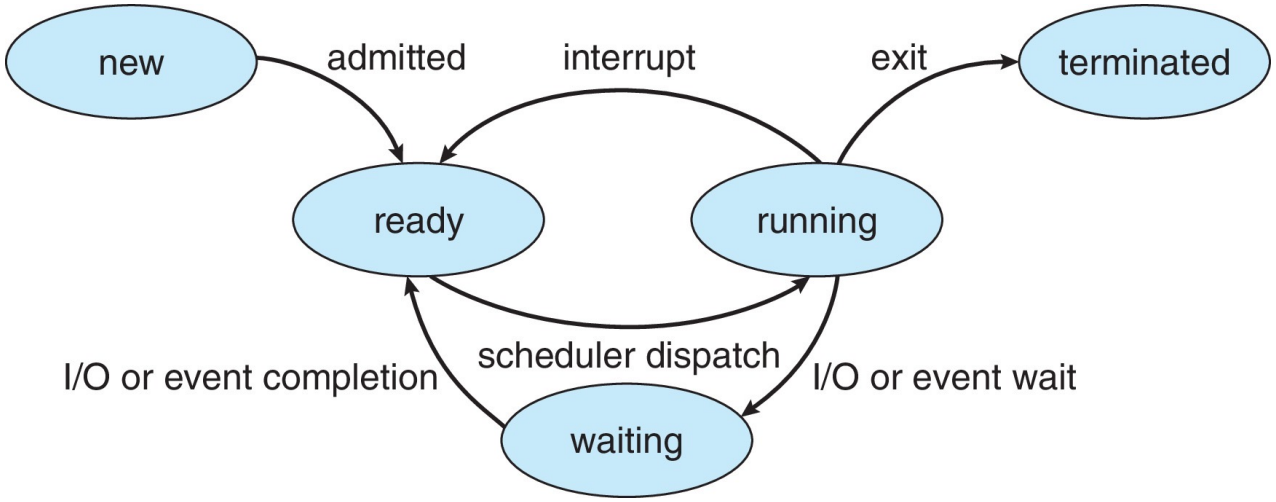


Program Execution (6/6)

```
L1: int g=0;

L2: void main() {
L3:     int *a = (int*) malloc(4);
L4:     char *b = “Hello World”;
L5:     foo(a);
L6:     g=*a;
L7: }

L8: void foo(int* b) {
L9:     *b = 20;
L10:}
```

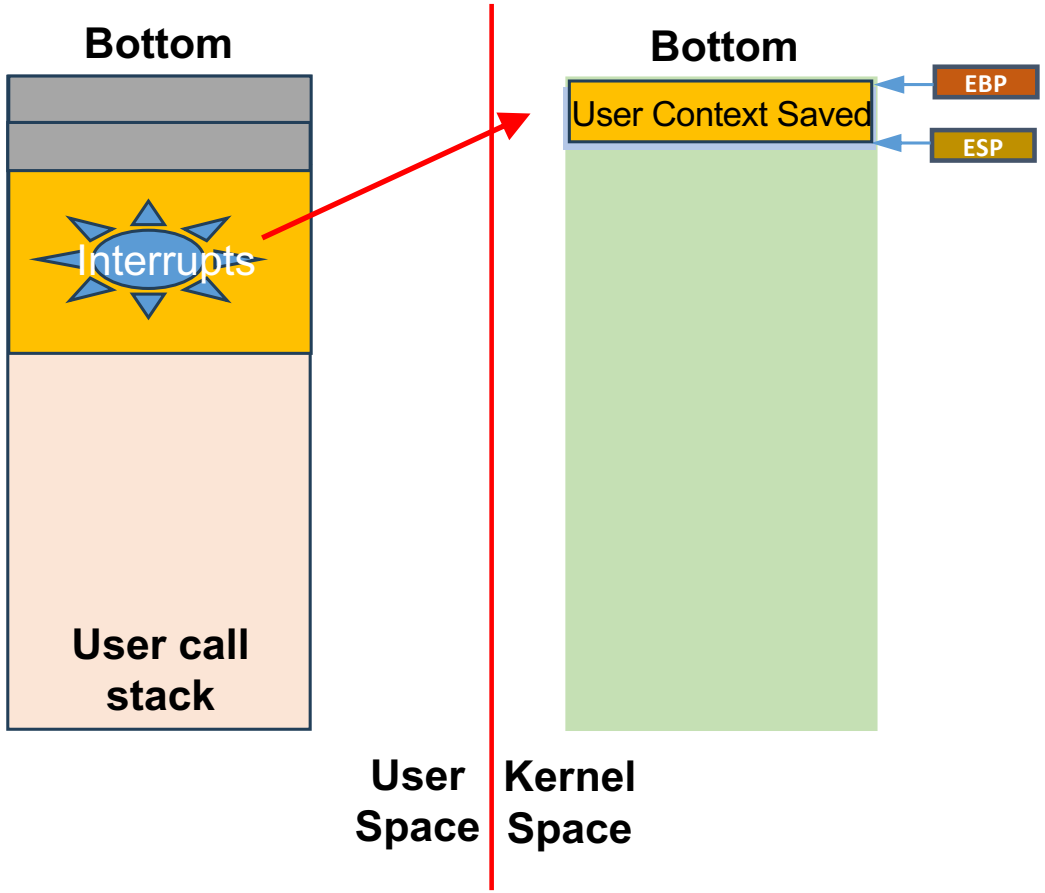


Process States



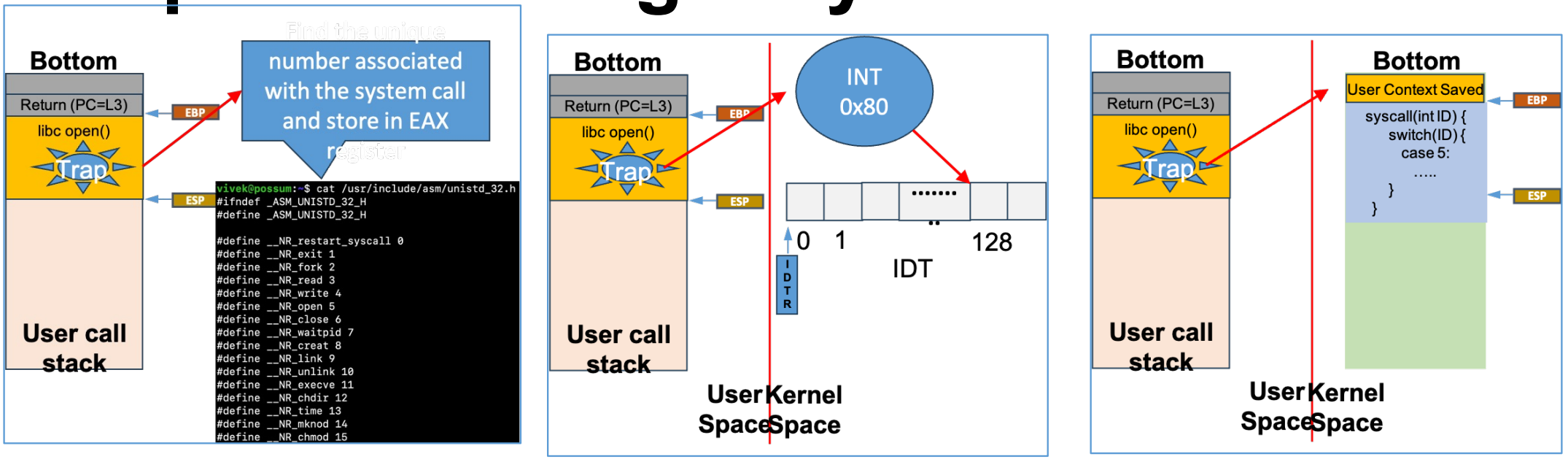
- As a process executes, it changes state
  - **New**: The process is being created
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
  - **Ready**: The process is waiting to be assigned to a processor
  - **Terminated**: The process has finished execution

Two Stacks for Each Processes



- OS maintains two stacks for each processes
  - User stack
  - Kernel stack
- Save user stack context (registers) in kernel mode stack while switching to kernel stack from user stack
  - Restore while jumping back to user stack from kernel stack

Steps for Making a System Call



- Each system call has a unique number associated that is declared inside the file unistd.h. Store this syscall number in the EAX register while the process is still executing inside user space
- Software interrupt (trap) is triggered after which syscall handler API funcptr is found in IDT index 128
- Switch to kernel stack, save user stack, followed by invoking syscall handler

Process Creation & Termination

```
int global=0;
int main() {
    if(fork() == 0) {
        global++;
        printf("Child process: Global=%d\n", global);
        char* args[2] = {"./fib", "40"};
        execv(args[0], args);
        printf("I should never print\n");
    } else {
        printf("I am the Parent process\n");
        int status;
        wait(&status);
    }
    printf("Global=%d\n", global);
    return 0;
}
```

Signals (Limited IPC)

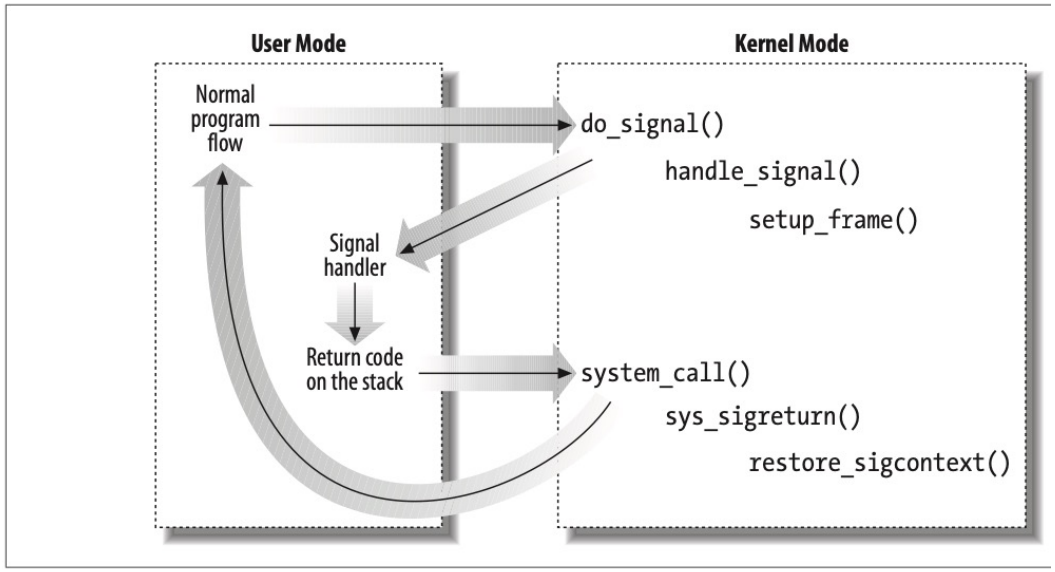


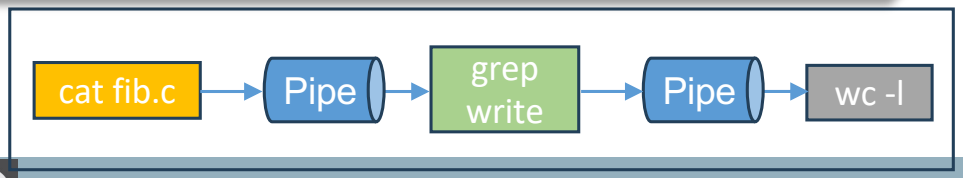
Figure 11-2. Catching a signal

- Each signal type has a default handler. A program can install its own handler for signals of any type
  - Except SIGKILL (default action is to exit the process) and SIGSTOP (default action is to suspend the process)
    - SIGSTOP meUnlike SIGKILL, SIGINT (Ctrl-c) is a polite way to terminate a process
- Process can use sigprocmask function to make changes to its signal mask (member in task_struct) if it wants to temporarily block a set of signals (not all signals can be blocked, e.g., SIGKILL and SIGSTOP)
- Process switches into kernel mode after receiving a signal
- In the absence of user signal handler, signal is handled in kernel mode, otherwise it is handled in the user mode
  - See the exact sequence as shown in the Figure when user has provided the signal handler (usermode → kernelmode → usermode → kernelmode → usermode)

Figure source: from the book "Understanding the Linux Kernel"

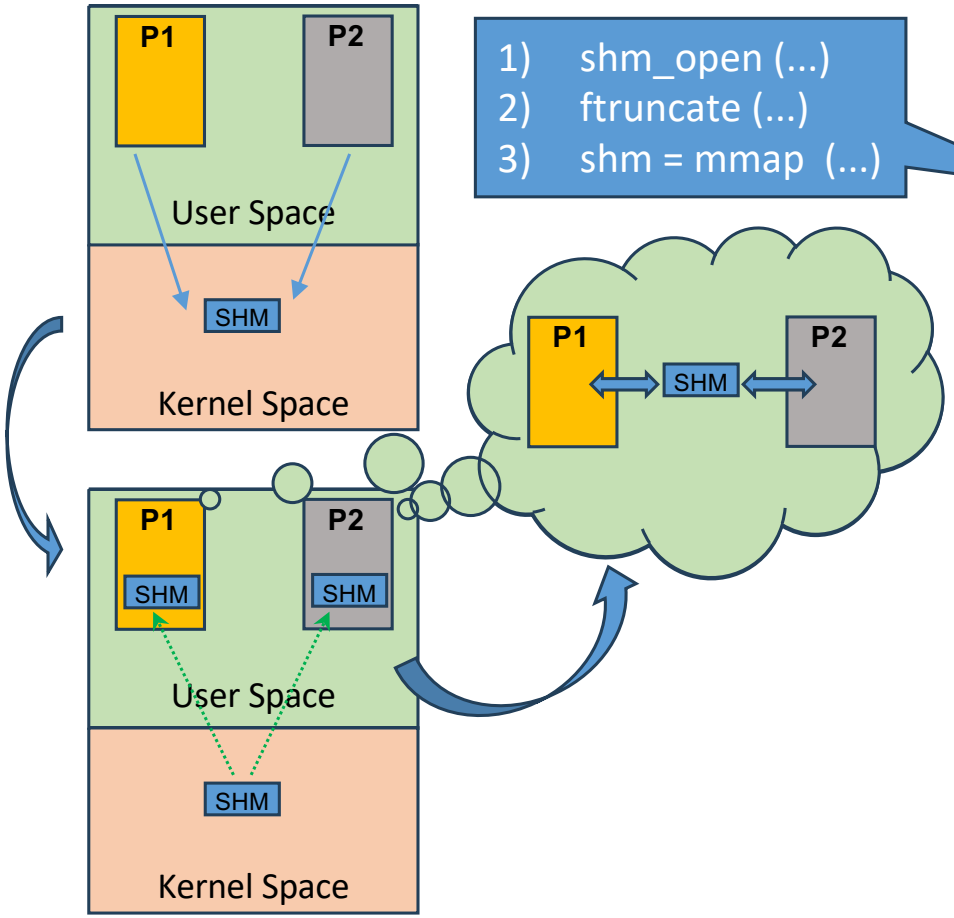
IPC Using Pipes

```
int main() {
    int fd[2], status;
    pipe(fd);
    if(fork() == 0) {
        /* Child process */
        close(fd[0]);
        char buff[] = "Hello my dear good Parent";

        write(fd[1], buff, sizeof(buff));
        exit(0);
    }
    /* Parent process */
    close(fd[1]);
    char buff[100];
    read(fd[0], buff, sizeof(buff));
    printf("My obedient child says: %s\n", buff);
    wait(NULL);
    return 0;
}
```



- Pipes (analogous to water pipe) is a unidirectional stream of data flowing from a source process to a destination process
  - Kernel buffer exposed to processes as a pair of file descriptors (readable end and writable end)
  - Data delivered in the same order as sent
  - Uses blocking IO and the writer process will block if the pipe is full
- We use it frequently on Shell
  - Better than using temporary files as pipes automatically clean up themselves unlike files that must be explicitly removed using the "rm" command on Shell

IPC Using POSIX Shared Memory



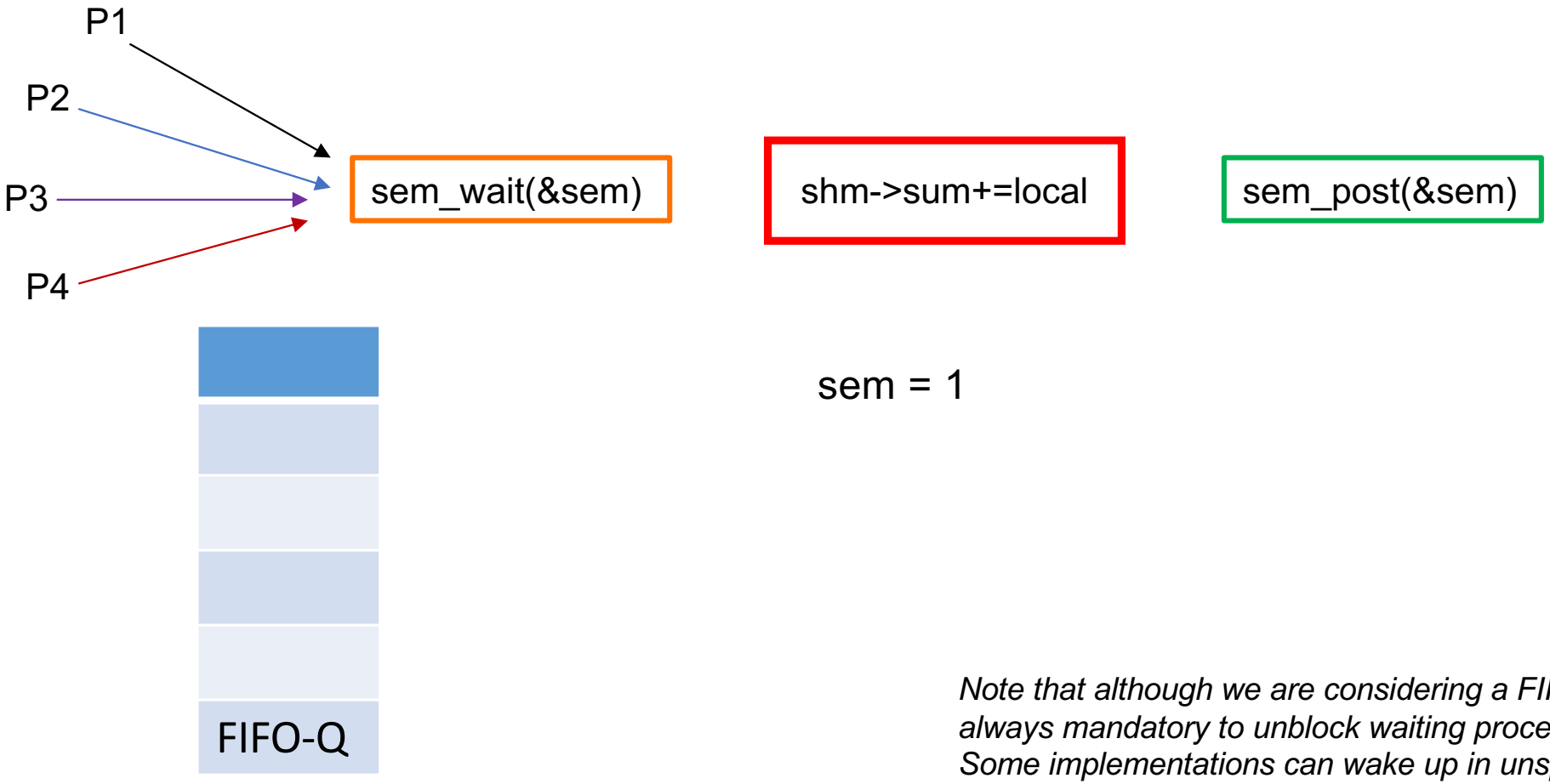
1) shm_open (...)
2) ftruncate (...)
3) shm = mmap (...)

```
typedef struct shm_t {
    int A[SIZE];
    int sum1;
    int sum2;
} shm_t;
```

```
int main() {
    shm_t* shm = setup();
    if(fork()==0) {
        int local=0;
        for(int i=0; i<SIZE/2; i++) shm->sum1+=shm->array[i];
        cleanup_and_exit();
    } else {
        int local=0;
        for(int i=SIZE/2; i<SIZE; i++) shm->sum2+=shm->array[i];
        wait(NULL);
    }
    int total = shm->sum1 + shm->sum2;
    cleanup();
    return 0;
}
```

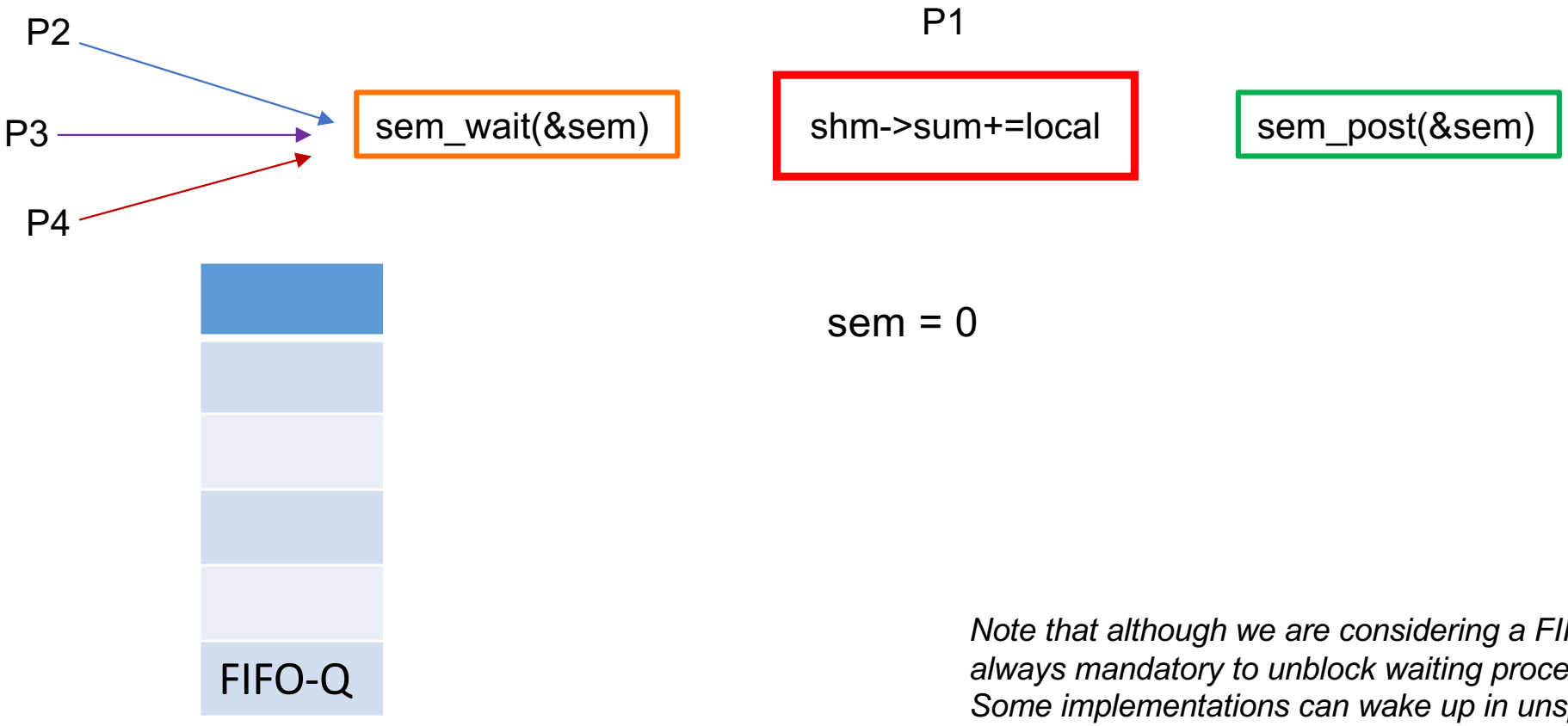
1) munmap (shm, ...)
2) close (...)
3) shm_unlink (...)

Semaphores (1/6)



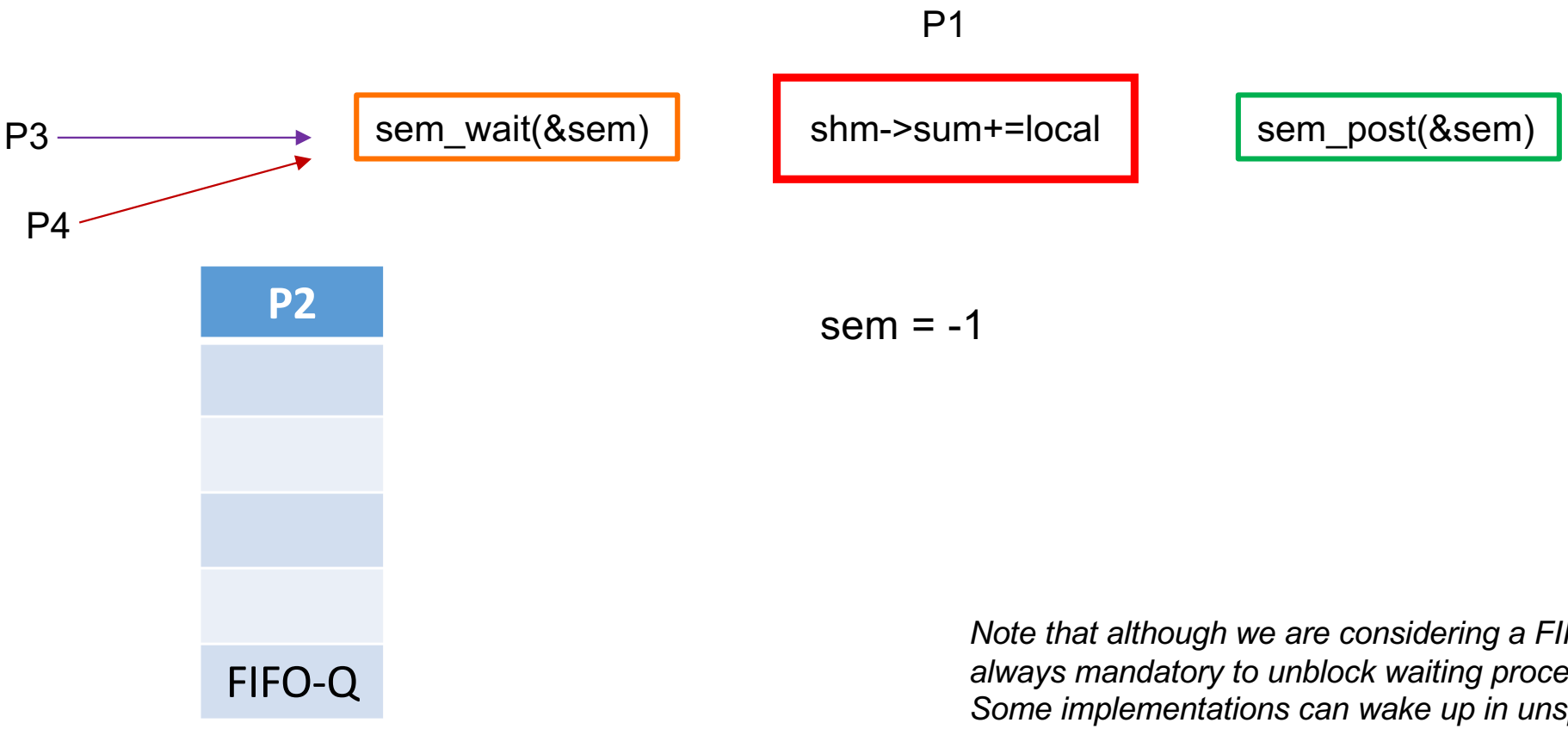
Note that although we are considering a FIFO order, it is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (2/6)



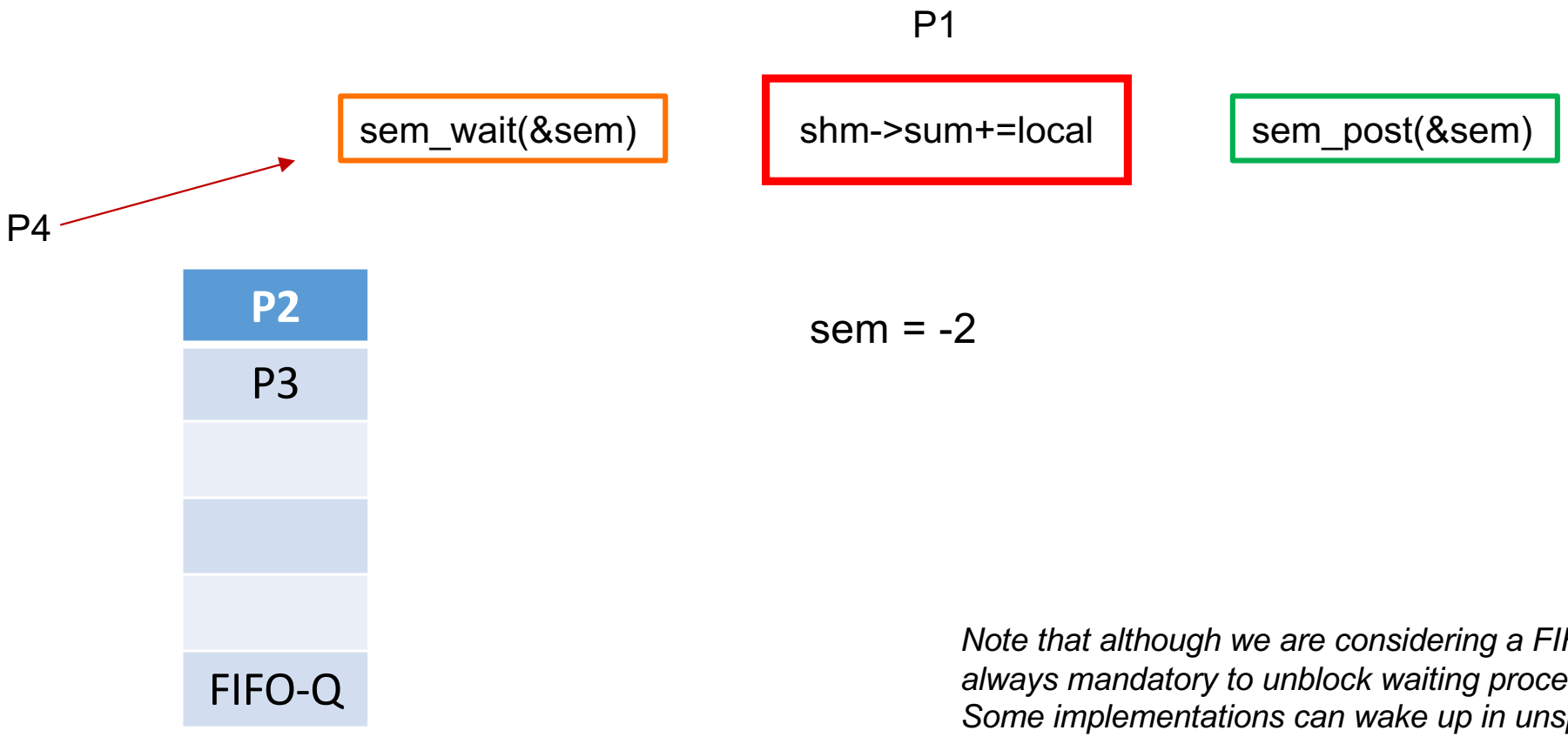
Note that although we are considering a FIFO order, it is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (3/6)



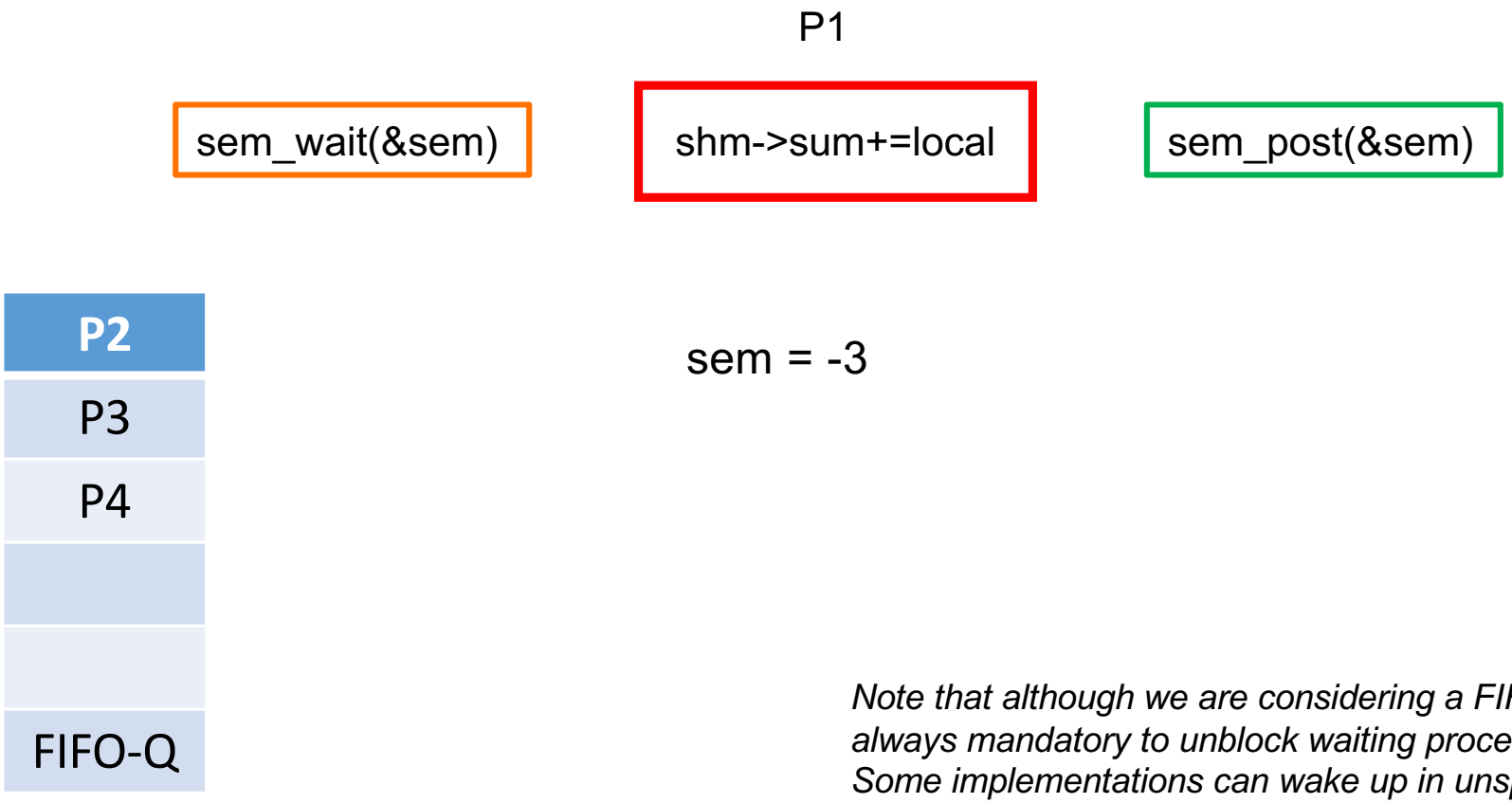
Note that although we are considering a FIFO order, it is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (4/6)



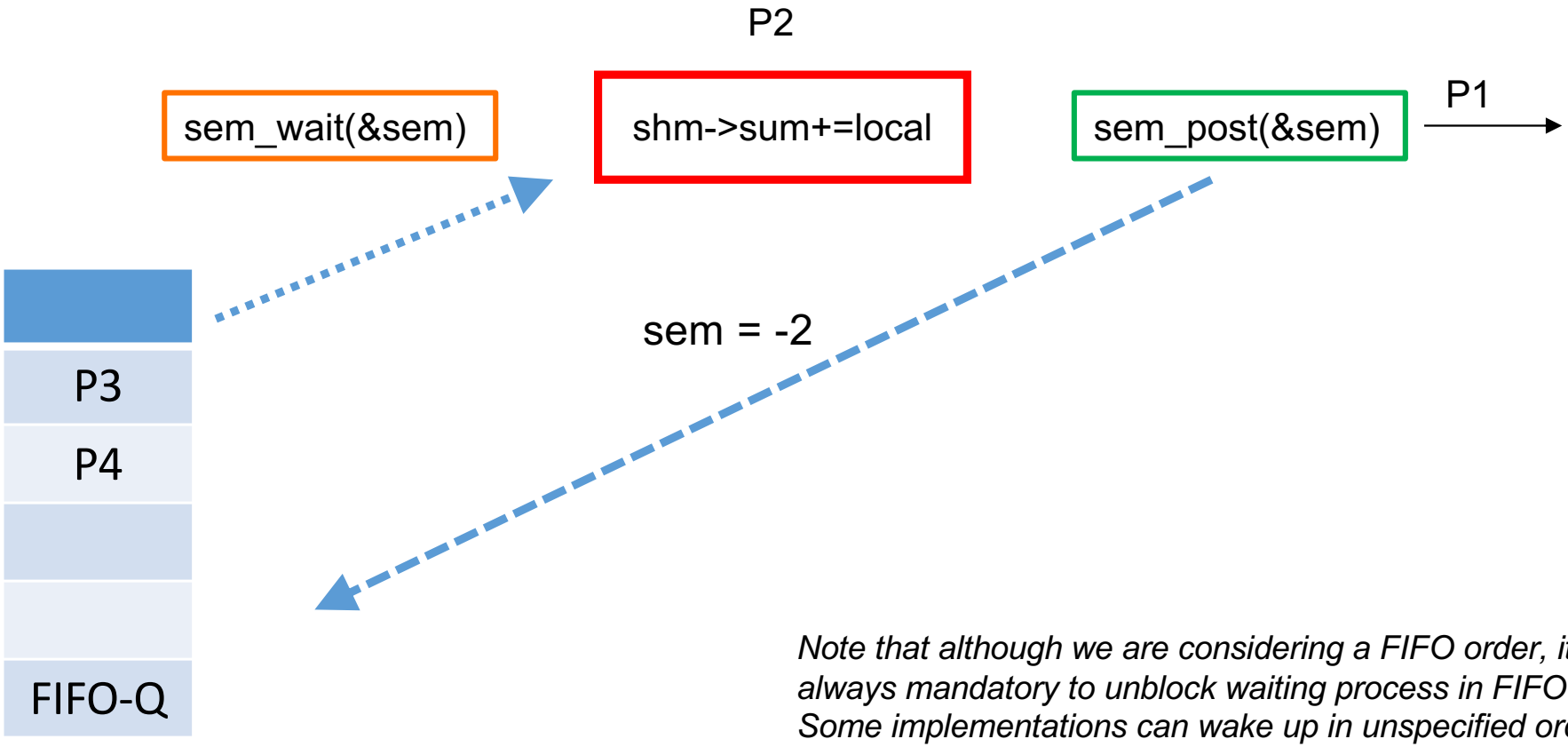
Note that although we are considering a FIFO order, it is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (5/6)



Note that although we are considering a FIFO order, it is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (6/6)



Note that although we are considering a FIFO order, it is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Producer Consumer Problem

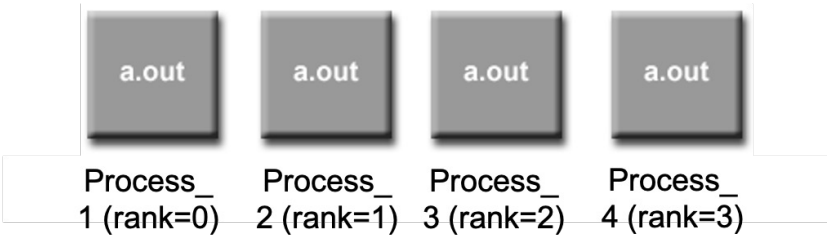
```
typedef struct cookiejar_t {
    int cookie;
    sem_t jar_empty;
    sem_t jar_full;
} cookiejar_t;

cookiejar_t* cookiejar;

int main() {
    cookiejar = setup();
    cookiejar->empty=1;
    sem_init(&cookiejar->jar_empty, 1, 1);
    sem_init(&cookiejar->jar_full, 1, 0);
    if(fork() == 0) { homer(); }
    if(fork() == 0) { marge(); }
    wait(NULL); // wait for Homer process
    wait(NULL); // wait for Marge process
    sem_destroy(&cookiejar->jar_empty);
    sem_destroy(&cookiejar->jar_full);
    cleanup();
    return 0;
}
```

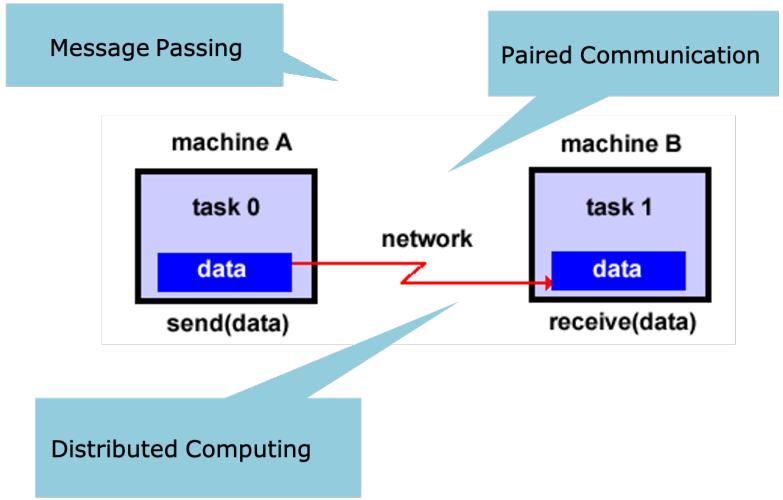
```
void homer() {
    for(int i=0; i<5; i++) {
        sem_wait(&cookiejar->jar_full);
        printf("Homer ate Cookie-%d\n", cookiejar->cookie);
        sem_post(&cookiejar->jar_empty);
    }
    cleanup_and_exit();
}
```

```
void marge() {
    for(int i=0; i<5; i++) {
        sem_wait(&cookiejar->jar_empty);
        printf("Marge bake Cookie-%d\n", ++cookiejar->cookie);
        sem_post(&cookiejar->jar_full);
    }
    cleanup_and_exit();
}
```

Homer Process (CPU 0)		Sem (empty)	Sem (full)	Marge Process (CPU 1)	
Action	**Process State**	**Sem Value**	**Sem Value**	**Action**	**Process State**
Forked	Ready	1	0	Forked	Ready
wait(full)	Blocked	0	-1	wait(empty)	Running
	Blocked	0	-1	Bake Cookie-1	Running
	Ready	0	0	post(full)	Running
Eat Cookie-1	Running	-1	0	wait(empty)	Blocked
post(empty)	Running	0	0		Ready
wait(full)	Blocked	0	-1	Bake Cookie-2	Running
	Ready	0	0	post(full)	Running
…….				…….	

IPC in Distributed Memory






```
int main(int argc, char **argv) {
    int rank=0, nproc=4;
    MPI_Init(&argc, &argv);
    // 1. Get to know your world
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &nproc);
    int array[SIZE]; // initialized and assume (SIZE % nproc = 0)
    // 2. calculate local sum
    int my_sum = 0, chunk = SIZE/nproc;
    for (int i=rank*chunk; i<(rank+1)*chunk; i++) my_sum += array[i];
    // 3. All non-root processes send result to root processes (rank=0)
    if(rank > 0) {
        MPI_Send(&my_sum, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }
    else { // executed only at rank=0
        int total_sum = my_sum, tmp;
        for(int src=1; src<nproc; src++) {
          MPI_Recv(&tmp, 1, MPI_INT, src, 0, MPI_COMM_WORLD, NULL);
          total_sum += tmp;
        }
    }
    MPI_Finalize();
}
```

Where are we as of now

- **CSE231 Post Conditions**
  1. Students are able to create a Unix shell with complete clarity about process creation and process execution
  2. Students are able to write multi- threaded applications with synchronization primitives and ability to analyze effects of concurrency on process execution and correctness
  3. Students are able to analyze the impact of OS concepts, e.g. virtual memory, concurrency, on program execution and ability to fine-tune the program to run efficiently on a given OS
  4. Students are able to demonstrate deeper understanding of the Unix-like OSes and kernel programming

All the best for your exam !!

- Exam syllabus
  - Lectures **02-10**
- Exam schedule
  - Date: Monday (22nd September)
  - Time: 10:00am-11:00am (One hour only)
  - Venue: C102 LHC
- Exam pattern
  - Similar to quizzes
  - MCQ, subjective questions, and program writing